PROCESS MANAGEMENT SUBSYSTEM IN LINUX

What is a Process?

A program that is loaded into RAM for execution is called a **Process.**

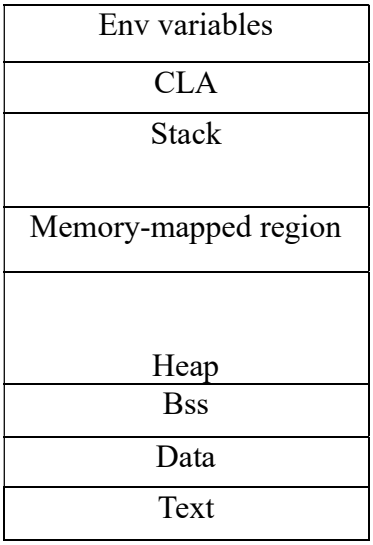
Program → present in HD

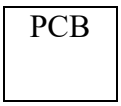
Compile (gcc)

Executable → present in HD

./a.out

Execution → loaded into RAM





Process Control Block (PCB):

All the information related to a process are stored in an object called **Process Control Block (PCB)**.

PCB is of type **struct task_struct** .

Why task_struct? The OS views a process/thread as a task.

What are the contents of PCB? The PCB structure has the following members:

- ✓ PID
- ✓ PPID
- ✓ Page table
- ✓ Signal Disposition table
- ✓ File Descriptor table
- ✓ Stack Pointer (SP)
- ✓ Program Counter (PC)
- ✓ Signal Mask
- ✓ Process state
- ✓ Pointer to kernel stack

Commands to check the contents of PCB:

- $ ps -ef
- $ ps -es
- $ top
- /proc file system

- PCBs are present as nodes in a doubly linked list.
- PCBs are arranged as queues – Running queue and waiting queue.
- Scheduler is a program. It is the head node of the linked list. The scheduler is responsible for scheduling the processes for execution.
- Types of schedulers:
    1. Long-term scheduler – (HD to RAM)
    2. Short-term schedule – (Ready queue to RAM)
    3. Medium-term scheduler – (RAM to HD) – swap out and swap in

CPU scheduling algorithms (used by the short-term scheduler):

- Round Robbin (RR)
- Priority-based
    1. Non-preemptive
    2. Preemptive
- Shortest Job First (SJF)
- First Come First Serve (FCFS)

CONTEXT SWITCHING

What is a context?

Context is the information that describes the current execution state of a task (process/thread).

What is context switching?

Context switching is the process of saving the context of the currently running process and restoring the context of another process. Thus, the process loaded into RAM will resume execution from where it left off.

Why context switching?

A CPU can execute only one instruction stream per core at a time.

When context switching takes place?

- When the time slice has expired.
- When a higher-priority task arrives
- Hardware interrupt
- Software interrupt (signal)

What is stored in the context area?

- ✓ Program counter (PC)
- ✓ Stack pointer (SP)
- ✓ General-purpose registers
- ✓ Status register/flags
- ✓ Process state

The flow of context switching:

Process1(user mode) ----interrupt--→ process1(kernel mode) → context area(P1) → process2 → context area(P2) → process2(kernel mode) → process1(user mode)

1st save – in kernel stack

2nd save – in PCB

CREATION OF A PROCESS

fork():

fork() is a system call to create a process from another process.

fork() returns twice:

- In child process – 0
- In parent process – PID of child process

The fork() system call is a libc wrapper that internally uses the sys_fork() kernel call (conceptually). In modern kernel, sys_fork() invokes kernel_clone() to create a new child process (task_struct).

Depending upon the scheduler, either the parent process executes first, or the child process.

Orphan Process :– parent terminates, but child process is still alive. The init process adopts the orphan process.

Zombie Process :– child dies, but the parent has not yet collected the exit status of the child process.

wait() system is used to avoid a child process becoming an orphan process or zombie process. wait() does two things:

1. Execution synchronization
2. Resource cleanup

vfork():

vfork() is also a system call similar to fork(). It creates a child process, but with a different motive. The two main agendas of vfork():

1. Parent and child processes share the same resource(address space)
2. Parent process suspends until the execution of child process

vfork() internally uses sys_vfork(), which again uses clone() to create a new child process.

- In vfork(), the parent process suspends until the child exits or execs.

the clone() system call:

clone() is a highly customizable and flexible system call used to create a new task (process or thread).

fork(), vfork(), and pthread_create() internally uses clone() to create a new task_struct (whether process or thread).

clone() provides flags which tells the OS what resources the newly created child process(task) should share with the parent